

目录

原生视频插件开发计划 1

概述 1

目标 1

架构设计 1

1. UDP 接收层（原生） 1

2. 解码层（NVDEC/CUVID） 2

3. 颜色转换与上传 2

4. Unity 桥接层 2

5. 日志与诊断 2

构建说明 2

依赖 2

目标产物 2

CMake 配置要点 3

开发任务 3

阶段一：原生 UDP 接收 3

阶段二：CUVID 解码 3

阶段三：颜色转换与互操作 3

阶段四：Unity 集成 3

阶段五：文档与构建 3

性能预期 3

当前状态 4

已完成 4

已知问题 4

解决方案 4

原生视频插件开发计划

概述

Native NVDEC + OpenGL 4.5 集成计划，实现 Ubuntu 24.04 + NVIDIA GPU 环境下的零拷贝视频解码路径。

目标

- 使用 NVDEC (Video Codec SDK) 通过 CUDA 固定功能单元解码 HEVC/H264 AnnexB 码流
- 零拷贝路径：UDP → 原生接收/组帧 → NVDEC 解码 (NV12) → CUDA 颜色转换 → PBO/GL 纹理 → Unity 通过原生插件事件采样纹理 ID
- 保留 C# ffmpeg 管道作为回退方案

架构设计

1. UDP 接收层（原生）

- 使用 recvmsg 在 AF_INET/AF_INET6 上接收
- 固定池缓冲区（C++ 预分配 slab，等效于 C# ArrayPool）
- 分片头格式：
 - frameId (u16 LE): 帧编号
 - sliceId (u16 LE): 分片编号
 - frameLen (u32 LE): 帧总长度
 - payload: AnnexB 分片数据
- 帧环形缓冲区按 frameId 索引

- 每帧有序分片映射
- 超时与溢出驱逐
- 丢弃旧帧策略保持低延迟
- 参数集缓存（VPS/SPS/PPS），IDR 到达时在解码前重发参数集

2. 解码层（NVDEC/CUVID）

- 创建 CUcontext + CUvideotxlock + CUvideoparser + CUvideodecoder
- 解析器回调：
 - on sequence → 初始化/重新初始化解码器
 - on picture → 入队解码
 - on display → 映射解码后表面
- 优先支持 HEVC；如检测到则允许 H264

3. 颜色转换与上传

- 映射 NVDEC 输出（NV12 格式）
- 使用 CUDA kernel 或 NPP 执行 NV12 → RGB8 转换
- CUDA-OpenGL 互操作：
 - 用 CUDA 注册 PBO/纹理
 - 将 RGB 写入 PBO
 - glTexSubImage 更新常驻纹理
- 双/三缓冲
- 存储最新纹理 ID 供 Unity 获取

4. Unity 桥接层

C 导出接口：

```
int nvp_init(int width, int height);
void nvp_push_udp(const uint8_t* ptr, int len);
int nvp_get_latest_texture();
void nvp_shutdown();
```

C# 封装（NativeVideoBridge）： - P/Invoke 调用导出函数 - 使用 GL.IssuePluginEvent 或 CustomTextureUpdateV2 绑定最新纹理 - 插件加载失败时回退到 ffmpeg 管道

5. 日志与诊断

- 高频路径仅保留计数器
- 每秒聚合日志输出
- 可选 verbose 标志用于转储解析器/解码器状态快照

构建说明

依赖

- CUDA Toolkit（含 Video Codec SDK 头文件）
- OpenGL 开发头文件
- CMake ≥ 3.22

目标产物

- LibNativeVideoPlugin.so 共享库
- 安装位置：Assets/Plugins/x86_64/

CMake 配置要点

```
find_package(CUDA REQUIRED)
find_package(OpenGL REQUIRED)

target_link_libraries(NativeVideoPlugin
    ${CUDA_LIBRARIES}
    nvcuvid
    ${OPENGL_LIBRARIES}
    dl
    pthread
)
```

开发任务

阶段一：原生 UDP 接收

- ☐ 实现原生 UDP 接收
- ☐ 实现帧环形缓冲区

阶段二：CUVID 解码

- ☐ 接入 CUVID 解析器/解码器
- ☐ 推送 AnnexB 数据
- ☐ IDR 前参数集重发

阶段三：颜色转换与互操作

- ☐ CUDA kernel 实现 NV12→RGB
- ☐ CUDA-GL 互操作到 PBO/纹理
- ☐ 暴露纹理 ID

阶段四：Unity 集成

- ☐ 命令缓冲区 / 插件事件将原生纹理 blit 到材质
- ☐ 暴露切换开关选择原生 vs ffmpeg 路径

阶段五：文档与构建

- ☐ 添加构建说明到 CHANGES.md
 - ☐ 确认依赖后完善 CMake 的 CUDA/GL 链接
-

性能预期

延迟阶段	FfmpegPipe+CUDA	NativeNvdec
UDP 组帧	50-80ms	50-80ms
解码	50-80ms	20-40ms
纹理上传	20-30ms	5-10ms
总延迟	~120-190ms	~75-130ms

当前状态

已完成

- ☒ SDK 头文件安装
- ☒ 动态加载机制实现
- ☒ 编译框架搭建

已知问题

- ☒ GL 互操作在主线程初始化失败（需延迟到渲染线程）

解决方案

修改插件逻辑，在渲染线程首次调用 `nvp_get_latest_texture()` 时才创建 GL 资源： 1. `nvp_init()`：只初始化 CUDA 和 NVDEC 解码器 2. `nvp_get_latest_texture()`：首次调用时初始化 PBO/Texture（此时有 GL 上下文）

文档更新日期：2026年2月9日